

DOCUMENTAÇÃO TÉCNICA

1. Documentação do Pacote Modelo

O pacote *modelo* define as entidades centrais do sistema da Cafeteria. As classes de cadastro (*CadastroAtendente*, *CadastroCliente*, *CadastroProduto*) mantêm listas de objetos e permitem operações de busca, inserção e remoção, representando associações diretas. A classe abstrata *Cliente* estabelece a base para os tipos de clientes, com atributos comuns e métodos que podem ser sobrescritos. A herança é aplicada em *ClienteStandard* e *ClienteVIP*, que especializam o comportamento de acumulação de XP e benefícios.

O polimorfismo é evidenciado na sobrescrita do método *acumularXp*, permitindo que cada tipo de cliente tenha regras próprias de fidelização. Além disso, o uso de exceções customizadas, como *PontosInsuficientesException*, garante um tratamento adequado de erros relacionados ao programa de pontos.

1.1 Estrutura de Classes do Pacote Modelo

Classe / Interface	Tipo	Responsabilidade / Relacionamento Principal
Atendente	Classe concreta	Representa os colaboradores que operam o sistema e realizam os atendimentos dos pedidos
CadastroAtendente	Classe de controle	Gerencia o ciclo de vida dos atendentes através de uma associação direta (lista de atendentes)
CadastroCliente	Classe de controle	Centraliza o gerenciamento e persistência temporária de instâncias da classe Cliente
CadastroProduto	Classe abstrata	Mantém a listagem e o catálogo geral de itens disponíveis na cafeteria (associação com Produto)
Cliente	Classe concreta	Superclasse que define a estrutura comum de dados dos clientes (Nome, CPF, saldo XP).
ClienteStandard	Classe concreta	Especialização de Cliente com regras padrão de acúmulo de pontos de fidelidade.
ClienteVIP	Classe concreta	Especialização de Cliente com benefícios estendidos; consome pontos e comunica-se com exceções de saldo.
Pedido	Classe concreta	Centraliza o processo de venda, conectando cliente atendente e os respectivos itens de consumo.

Produto	Classe concreta	Define as propriedades de cada item do cardápio (código, nome, preço e estoque).
---------	-----------------	--

1.2 Mapeamento de Relações e Pilares OO

- **Associações:** as classes de controle mantêm listas fortemente tipadas de suas respectivas entidades. A classe *Pedido* manifesta uma alta multiplicidade de associações estruturais, vinculando-se simultaneamente a um *Cliente*, um *Atendente*, uma lista de *ItemPedido* e referências de *Produto*.
- **Herança:** implementada de forma hierárquica a partir da classe abstrata *Cliente* para as subclasses concretas *ClienteStandard* e *ClienteVIP*, maximizando o reaproveitamento de código e padronizando as assinaturas de dados.
- **Polimorfismo:** expressando por meio da sobrescrita (*override*) do método *acumularXp(valorGasto)*. Enquanto a lógica padrão adiciona pontos proporcionalmente lineares, a versão da subclasse VIP aplica um multiplicador de acúmulo em dobro:
 - Cliente Standard: $XP = valorGasto \times 1.0$
 - Cliente VIP: $XP = valorGasto \times 2.0$

2. Documentação do Pacote Serviço

O pacote *servico* define regras de negócio e serviços auxiliares da Cafeteria. As interfaces *ServicoVenda* e *Promocional* permitem a implementação polimórfica de diferentes estratégias de venda e aplicação de descontos, aumentando a flexibilidade do sistema.

A classe *GerenciadorEstoque* centraliza a validação e atualização de estoque, utilizando exceções customizadas para garantir consistência. Já a classe *CalculadoraPontos* aplica regras de fidelização, diferenciando o acúmulo de pontos conforme o tipo de cliente. Esse uso de polimorfismo reforça a extensibilidade e a personalização da experiência do usuário. Assim, o pacote serviço conecta modelo de dados (*Pedidos*, *Produto*, *Cliente*) às regras de negócio, garantindo robustez e escalabilidade.

2.1 Dinâmica de Interfaces e Polimorfismo

O método *Pedido.finalizarPedido()* utiliza o polimorfismo de interface ao interagir diretamente com a interface *Promocional*. Desta forma, o fluxo de fechamento de vendas torna-se independente da implementação concreta do desconto (seja ele baseado em cupons sazonais, descontos por convênio ou promoções do dia), garantindo a extensibilidade arquitetural sem a necessidade de alterar a classe controladora principal do pedido.

Adicionalmente, a classe *CalculadoraPontos* executa um polimorfismo baseado em instâncias através da validação dinâmica de tipos, adaptando o comportamento dos cálculos e bonificações dependendo se o objeto recebido em tempo de execução corresponde a um perfil Standard ou VIP (verificação via *instanceof*).

3. Documentação do Pacote Exceção

O pacote *excecao* concentra as exceções customizadas utilizadas no sistema da Cafeteria. Ambas são implementadas como *unchecked exceptions*, garantindo maior flexibilidade no código e evitando a necessidade de tratamento obrigatório em todos os métodos.

A classe *EstoqueInsuficienteException* assegura que operações de venda não sejam concluídas sem estoque disponível, protegendo a integridade dos dados. Já a classe *PontosInsuficientesException* garante que o programa de fidelidade seja utilizado corretamente, evitando que clientes VIP realizem resgates sem saldo suficiente. Essas exceções reforçam a robustez do sistema, permitindo mensagens claras e específicas para cada tipo de erro.

3.1 Justificativa Técnica das Exceções Customizadas

Exceção Customizada	Hierarquia Nativa	Justificativa de Arquitetura e Uso
<code>EstoqueInsuficienteException</code>	<code>RuntimeException (Unchecked)</code>	Disparada pelo <code>GerenciadorEstoque</code> se um item pedido exceder o saldo atual. Sendo uma <i>unchecked exception</i> , o código de interface ou camadas intermediárias não fica poluído com blocos excessivos de try-catch, permitindo uma captura centralizada.
<code>PontosInsuficientesException</code>	<code>RuntimeException (Unchecked)</code>	Lançada pelo método de resgate em <i>ClienteVIP</i> se o saldo de XP for menor que o necessário. Protege as assinaturas dos métodos de serviço contra acoplamentos rígidos a exceções checadas, facilitando a manutenção.

Nota Arquitetural: A escolha por exceções não-checadas (*Unchecked Exceptions*) alinha-se às práticas modernas de desenvolvimento em Java, delegando a responsabilidade de prevenção aos validadores de negócio e mantendo as regras de controle de fluxo limpas e desacopladas de declarações compulsórias de infraestrutura.

4. Conclusão

A arquitetura orientada a objetos proposta para o sistema da Cafeteria apresenta um alto índice de coesão e baixo acoplamento entre seus pacotes constitutivos:

- O pacote **modelo** atua como a fundação de dados estruturada sob regras rígidas de herança e encapsulamento.
- O pacote **servico** isola os algoritmos operacionais e regras flutuantes de mercado por meio de interfaces abstratas.
- O pacote **excecao** provê a segurança transacional necessária para mitigar estados inconsistentes de inventário e fidelização.

